Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency. Develop a program that executes the equivalent of the shell command: ls -l | grep ".c" using fork(), pipe(), dup2(), and exec() system calls.

1. Producer–Consumer Using Anonymous Pipes

```
adithya@LAPTOP-07A78KDI:~/OS$ nano producer_consumer.c
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <string.h>

#define NUM_MESSAGES 10000

int main()
{
    int pipefd[2];
    pid_t pid;
    char buffer[100];

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    /* Create child process */
    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(1);
    }

    /* Child process - Consumer */
    if (pid == 0)
    {
        close(pipefd[1]);

        int total_bytes = 0;
        int bytes_read;

        while ((bytes_read = read(pipefd[0], buffer, sizeof(buffer))) > 0)
        {
            total_bytes += bytes_read;
        }

        printf("Consumer received %d bytes\n", total_bytes);

        close(pipefd[0]);
    }
```

```c
    /* Parent process - Producer */
    else
    {
        close(pipefd[0]);

        struct timeval start, end;

        gettimeofday(&start, NULL);

        for (int i = 0; i < NUM_MESSAGES; i++)
        {
            sprintf(buffer, "Message %d\n", i + 1);

            write(pipefd[1], buffer, strlen(buffer));
        }

        close(pipefd[1]);

        wait(NULL);

        gettimeofday(&end, NULL);

        double time_taken =
            (end.tv_sec - start.tv_sec) * 1000000.0 +
            (end.tv_usec - start.tv_usec);

        printf("Producer sent %d messages\n", NUM_MESSAGES);
        printf("Communication time: %.2f microseconds\n", time_taken);

        if (time_taken > 0)
        {
            double throughput =
                (NUM_MESSAGES * 10.0) / time_taken;

            printf("Approximate throughput: %.2f MB/s\n",
                   throughput);
        }
    }

    return 0;
}
```

```
adithya@LAPTOP-07A78KDI:~/OS$ gcc producer_consumer.c -o producer_consumer
adithya@LAPTOP-07A78KDI:~/OS$ ./producer_consumer
Consumer received 128894 bytes
Producer sent 10000 messages
Communication time: 8188.00 microseconds
Approximate throughput: 12.21 MB/s
adithya@LAPTOP-07A78KDI:~/OS$
```

2. Implement `ls -l | grep ".c"` Using `fork()`, `pipe()`, `dup2()`, and `exec()`

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    /* Create first child */
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        exit(1);
    }

    if (pid1 == 0)
    {
        /* Child 1 - ls */

        close(pipefd[0]);

        /* Redirect stdout to pipe */
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp");
        exit(1);
    }
```

```c
    /* Create second child */
    pid2 = fork();

    if (pid2 < 0)
    {
        perror("fork");
        exit(1);
    }

    if (pid2 == 0)
    {
        /* Child 2 - grep */

        close(pipefd[1]);

        /* Redirect stdin from pipe */
        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp");
        exit(1);
    }

    /* Parent */
    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}
```

```
adithya@LAPTOP-07A78KDI:~/OS$ nano ls_grep.c
adithya@LAPTOP-07A78KDI:~/OS$ gcc ls_grep.c -o ls_grep
adithya@LAPTOP-07A78KDI:~/OS$ ./ls_grep
-rwxr-xr-x 1 adithya adithya 16552 Aug  8 09:21 client
-rw-r--r-- 1 adithya adithya  1348 Aug  8 09:21 client.c
-rw-r--r-- 1 adithya adithya   226 Jul 28 03:43 file.c
-rw-r--r-- 1 adithya adithya   986 Aug 14 08:59 hello.c
-rw-r--r-- 1 adithya adithya  1737 Aug 14 09:30 keyboard_shell.c
-rw-r--r-- 1 adithya adithya  1738 Aug 14 09:29 keyboard_shell.c.save
-rw-r--r-- 1 adithya adithya  1179 Sep 12 08:18 ls_grep.c
-rw-r--r-- 1 adithya adithya  2559 Aug 29 10:08 memory.c
-rw-r--r-- 1 adithya adithya  3201 Aug 29 10:07 parser.c
-rwxr-xr-x 1 adithya adithya 16264 Aug  8 08:37 practical2
-rw-r--r-- 1 adithya adithya  1420 Aug  8 08:52 practical2.c
-rw-r--r-- 1 adithya adithya  1218 Aug  8 08:48 practical3.c
-rwxr-xr-x 1 adithya adithya 16344 Aug  8 08:56 practical4
-rw-r--r-- 1 adithya adithya  1787 Aug  8 08:59 practical4.c
-rwxr-xr-x 1 adithya adithya 16432 Aug  8 09:15 practical5
-rw-r--r-- 1 adithya adithya  1548 Aug  8 09:13 practical5.c
-rwxr-xr-x 1 adithya adithya 16296 Aug 14 09:05 process_demo
-rw-r--r-- 1 adithya adithya  1072 Aug 14 09:07 process_demo.c
-rwxr-xr-x 1 adithya adithya 16296 Aug 26 06:45 process_states
-rw-r--r-- 1 adithya adithya  2699 Aug 26 06:59 process_states.c
-rwxr-xr-x 1 adithya adithya 16464 Sep 12 08:13 producer_consumer
-rw-r--r-- 1 adithya adithya  1732 Sep 12 08:13 producer_consumer.c
-rw-r--r-- 1 adithya adithya  1728 Aug  8 09:20 server.c
-rw-r--r-- 1 adithya adithya   526 Aug 14 09:24 shell.c
-rw-r--r-- 1 adithya adithya    40 Aug  8 08:38 source.txt
-rw-r--r-- 1 adithya adithya  1802 Aug 29 09:48 tokenizer.c
-rw-r--r-- 1 adithya adithya   698 Aug 26 07:01 zombie.c
adithya@LAPTOP-07A78KDI:~/OS$
```